Robotics Experiment Report: Arduino Nano 33 BLE and ROS2 Serial Communication

Team: Team 3 Abdullay Sanoe ; Chibuike Okonkwo ; Mohamed Islam Maaroufi
Course: Robotics – Introduction to ROS2
Institution: TH Köln
Instructor: Prof. Dr. Elena Algorri
Date: 29th October, 2026

1. Introduction and Objectives

The objective of this experiment was to build a small mobile robot and control its movement using the publisher and subscriber architecture of the Robot Operating System 2 (ROS2). The project combined hardware assembly, Arduino programming, serial communication, and ROS2 Python programming.

The main objective of the task was to send movement commands through ROS2 and forward them to an Arduino Nano 33 BLE using serial communication. Deliverables such as the Arduino motor control and the ROS2 serial node template were provided to complete the task. The loaded code from the Arduino Nano 33 BLE took advantage of the board’s digital pins to perform Pulse Width Modulation (PWM) which drives the robot's servomotors.

The experiment also involved researching varying means of remote access to the Raspberry Pi from a laptop to establish a reliable, wireless control interface. Because the robot operates dynamically, a tethered wired connection was impractical. Therefore, the team evaluated both command-line and graphical remote desktop solutions. This included establishing a Secure Shell (SSH) connection via Windows PowerShell or Visual Studio Code to execute text-based ROS2 commands, as well as using graphical remote access software such as VNC Viewer. These wireless methods allowed the team to remotely execute commands, monitor the ROS2 nodes, and control the robot's hardware without needing a physical monitor or keyboard attached to the mobile chassis.

2. Hardware Components and Assembly

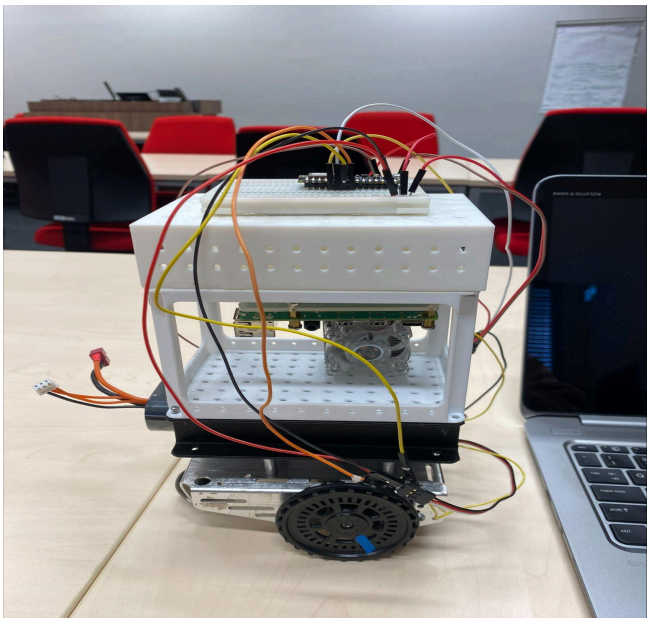
The robot was assembled using the components provided in the robotics kit:

- Robot chassis and servomotors
- LiPo battery
- DC-DC voltage regulator
- Battery pack
- Raspberry Pi
- Arduino Nano 33 BLE
- Breadboard
- Assortment of wires

The chassis provides the physical structure of the robot, while the servomotors enable movement. The Raspberry Pi acts as the main computer for running ROS2, and the Arduino receives serial commands to control the motors.

Assembly procedure:

The major assembly of the Robot was provided to the group in the form of the robot chassis which included the attached voltage regulator assembly, the two continuous servo motors, and the 3d printed mounting brackets and screws. The team assembled the unassembled components using guidance from literature and documentation provided. The Arduino Nano 33 BLE was attached to the breadboard which was also attached to the raspberry pi 3D printed housing using double sided tape. The voltage regulator wires were attached to power and ground to power the servo motors while signal wires were sent connected from the servos to the PWM pins of the Arduino Nano.



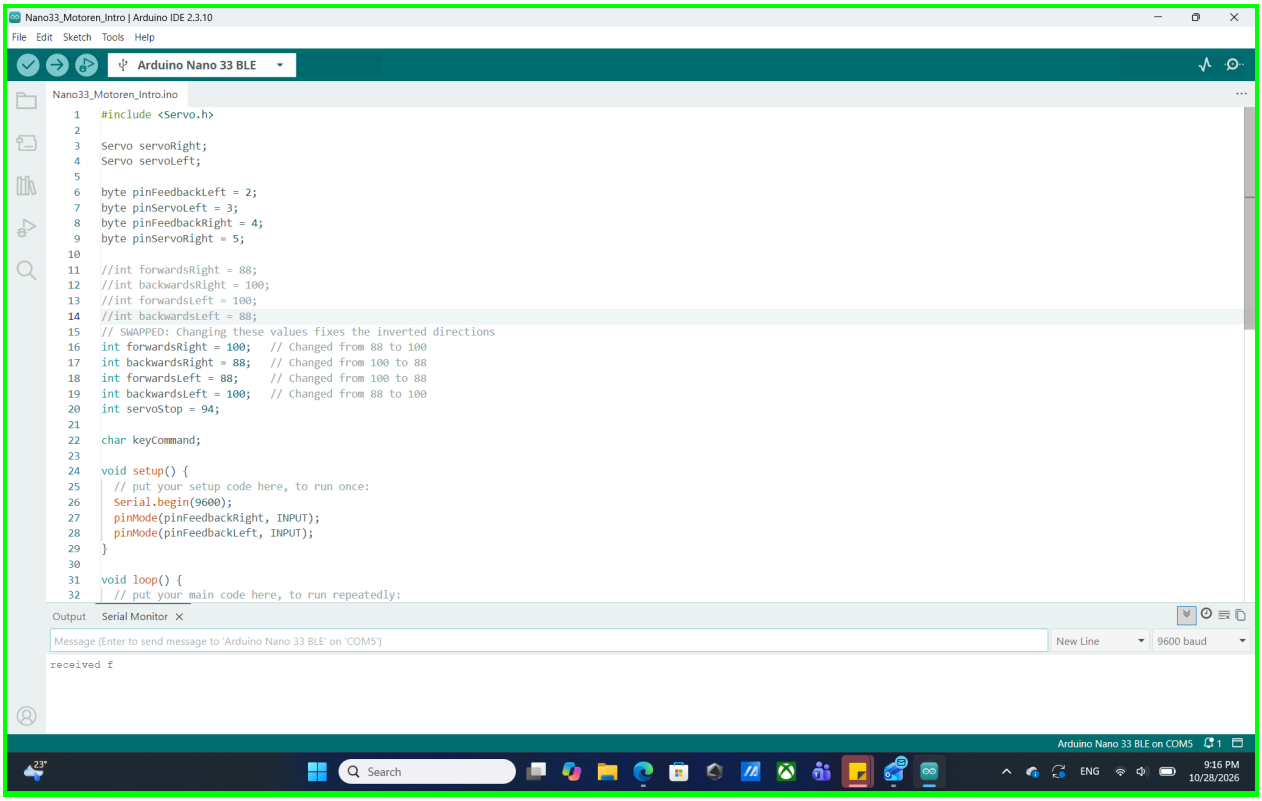
3. Arduino Programming and Initial Motor Test

The Arduino was programmed using the sketch provided in the laboratory materials. Before integrating it with ROS2, the motor-control functionality was tested using the Arduino Serial Monitor.

The commands specified in the assignment were:

- f – move forward
- b – move backward
- s – stop

Test results:



Firstly the team tested the servo motors alone with the Arduino Nano serial monitor to test how the code functions itself. Team members ran into issues with the board being visible in the sketch. A peripheral needed to be downloaded for the Arduino Nano 33 BLE specifically. Once downloaded the team ran the Nano33_Motoren_Intro.ino file. It was discovered that when the command inputs were given in the serial monitor the reverse of the intended behavior was shown. To rectify this, the team swapped the motor control values in the sketch.

4. ROS2 Python Node and Serial Communication

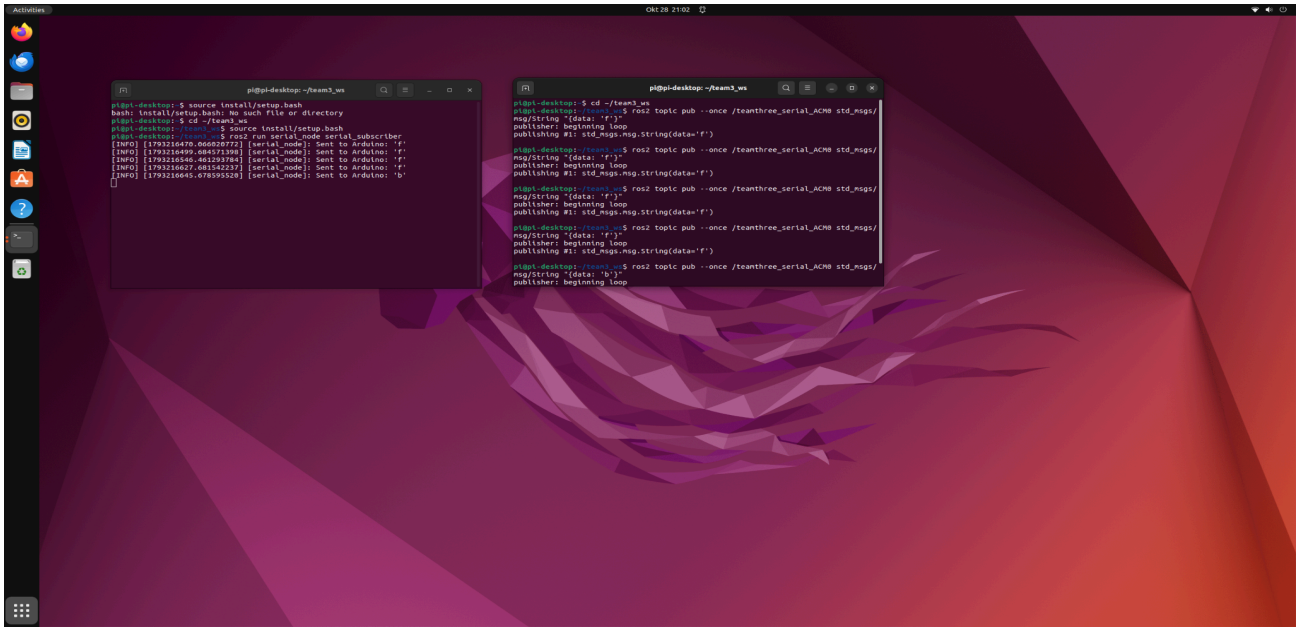
A ROS2 Python package named `serial_node` was used to implement the communication between ROS2 and the Arduino. The required node was named `serial_node_ACM0.py`.

The node subscribes to a ROS2 topic containing String messages. The topic name follows the assignment's naming convention, using the team's name in place of `algorri`, for example `/teamthree_serial_ACM0`.

When a message is received, the node extracts the command and sends it through the serial port to the Arduino. The Arduino then processes the command and controls the servomotors.

The communication sequence is:

1. A ROS2 message is published to the designated topic.
2. The Python node receives the message through its subscription.
3. The node sends the command over the serial connection.
4. The Arduino processes the command.
5. The servomotors respond to the command.



5. Remote Connection to the Raspberry Pi

The Raspberry Pi was used to run the ROS2 application. Remote access was configured so that the Raspberry Pi could be accessed from a laptop, preferably through Visual Studio Code and SSH.

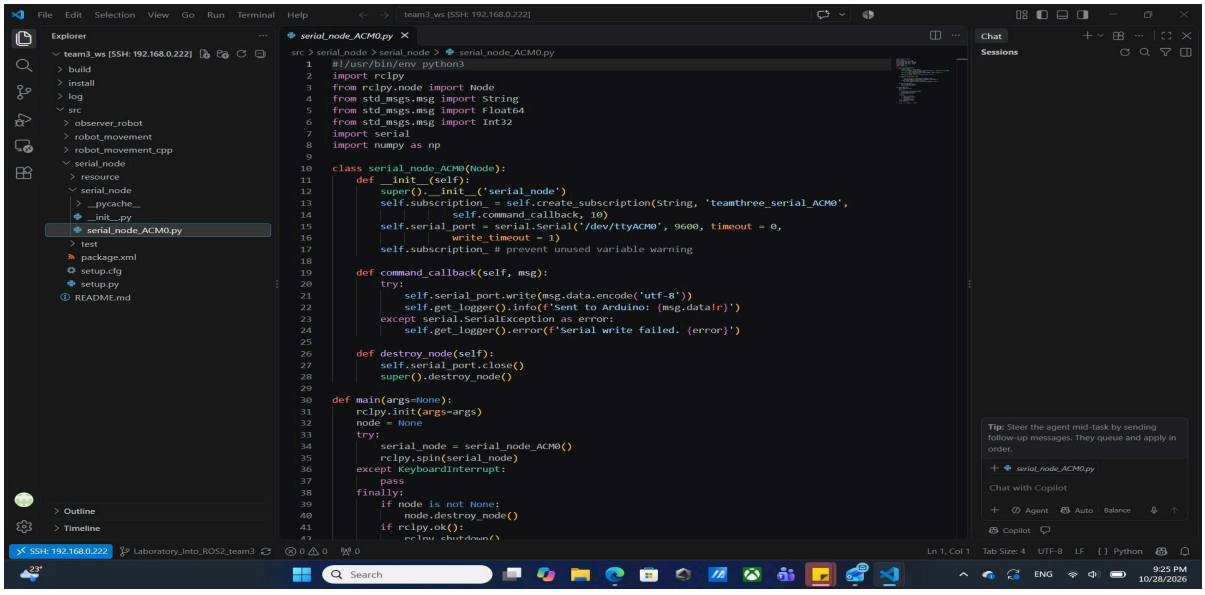
Connection method used: To establish the connection, the team connected both the laptop and the Raspberry Pi to a dedicated mobile hotspot to bypass the university network's client isolation restrictions. Once connected, we used the terminal command hostname -I to find the Raspberry Pi's IP address.

We then opened Visual Studio Code and used the "Remote - SSH" extension. We added a new SSH host using the command `ssh pi@[IP_Address]` and entered the password when prompted.

Network difficulties: Initially, we attempted to connect using the university Wi-Fi, but the laptop could not ping the Raspberry Pi due to network security policies. Switching to a mobile hotspot resolved this issue instantly, providing a stable, localized network for the SSH connection.

Connection procedure:

1. Connect the Raspberry Pi and laptop to a suitable network.
2. Enable and verify SSH on the Raspberry Pi.
3. Identify the Raspberry Pi's IP address.
4. Configure the Remote - SSH extension in Visual Studio Code.
5. Connect to the Raspberry Pi using its username and IP address.
6. Open the ROS2 workspace and run the required commands remotely.



6. Experiment and Results

The experiment was designed to verify that ROS2 messages could be transmitted to the Arduino and used to control the robot's movement.

Command	Expected behaviour	Actual observation
`f`	Robot moves forward	Moves forward
`b`	Robot moves backward	Moves backward
`s`	Robot stops	Movement Stops

For experimentation the team tested functionality of the robot in stages. When verified that the serial connection functioned as stated in another section of this report, the team moved on to testing the functionality of the serial_node subscriber node receiving the message data and then sending it to the arduino. To test this without physical hardware, the ROS package serial_node was created. In the second serial_node folder inside the src folder, the serial_node_ACM0.py script was created to listen for a /teamthree_serial_ACM0 topic of type string. A single string message was then published to the topic using the command terminal. The team then monitored the Arduino serial monitor to ensure that the message was received. Once the parts of functionality were working separately the team integrated the two parts and verified that the same message could be sent and could move the motor when all systems were powered. The following detailed steps are included below:

Create the workspace

A ROS 2 workspace is a directory containing your packages. We named ours `team3_ws` :

cd ~

mkdir -p ~/team3_ws/src

cd ~/team3_ws/src

Create a Python package/Create the `serial_node` package

From `~/team3_ws/src`, enter:

ros2 pkg create --build-type ament_python serial_node --dependencies rclpy std_msgs

This creates the package and its standard ROS 2 Python files.

The dependencies mean:

- `rclpy`: lets your Python script create and run a ROS 2 node.
- `std_msgs`: provides the `String` message type your node subscribes to

Install the serial library

Your script imports Python's `serial` module. The Ubuntu package that supplies it is `python3-serial`.

Run:

sudo apt update

sudo apt install python3-serial

Add your Python script (Create the script file)

Enter:
cd ~/team3_ws/src/serial_node/serial_node
nano serial_node_ACM0.py

This opens a terminal text editor called Nano.

We then Copy and paste or write our code into Nano:

What the script does

1. Creates a ROS 2 node named `serial_node`.
2. Subscribes to `algorri_serial_ACM0`.
3. Opens the Arduino's serial port at `9600` baud.
4. When a `String` message arrives, encodes it as UTF-8 and writes it to the serial port.
5. Closes the serial port when the node shuts down

Configure the executable entry point

ROS 2 needs to know how to launch our Python function using `ros2 run`.

Go to the outer package directory:

cd ~/team3_ws/src/serial_node

nano setup.py

Build the package

Run:

cd ~/team3_ws

source /opt/ros/iron/setup.bash

colcon build --packages-select serial_node --symlink-install

Wait for the build to finish. Ideally, the output ends with:

Summary: 1 package finished

Source the workspace

After the build succeeds, run:

source ~/team3_ws/install/setup.bash

To load the workspace

Launch the node

In Terminal 1, run:

source /opt/ros/iron/setup.bash

source ~/team3_ws/install/setup.bash

```
ros2 run serial_node serial_subscriber
```

The experiment was successful because it was verified that the robot was able to move once movement commands were published in the terminal.

7. Challenges and Solutions

1. **Reversed motor direction.** When commands were typed into the Serial Monitor, the robot moved in the opposite direction to the one intended (`f` moved it backward and `b` forward). The team resolved this by swapping the motor control values in the sketch, after which the behaviour matched the commands.
2. **No connection to the Raspberry Pi over the university network.** The laptop could not ping the Raspberry Pi on the university Wi-Fi because of network security policies (client isolation). Connecting both devices to a dedicated mobile hotspot solved the problem immediately and provided a stable local network for SSH.
3. **Testing the system in stages.** To locate faults more easily, the team tested each part separately. First the motors were tested through the Serial Monitor. Then the ROS2 subscriber node was tested by publishing a single String message from the terminal while watching the Arduino Serial Monitor. Only after both worked on their own were they integrated and tested with all systems powered.

8. Conclusion

This experiment provided practical experience with ROS2, Python nodes, topic subscriptions, serial communication, Arduino programming, and mobile robot hardware.

It demonstrated how high-level commands sent through ROS2 can be passed to a microcontroller to control a robot's movement.

The main learning outcome was an understanding of how software running on a Raspberry Pi can communicate with an Arduino to control physical hardware.